5. Decision Tree Model: High Training Accuracy but Poor Test Performance

Problem Analysis:
A decision tree performs very well on the training data but poorly on unseen (test) data. This indicates that the model has **poor generalization** and is likely **overfitting**.

Possible Causes:

1. **Overfitting:** The tree memorizes the training data instead of learning general patterns.
2. **Deep Tree Structure:** A very deep tree creates many branches that fit noise in the training data.
3. **Noisy Data:** Incorrect, missing, or inconsistent data can confuse the model.
4. **Irrelevant Features:** Unimportant features may reduce prediction accuracy.
5. **Small Training Dataset:** Limited data may not represent real-world patterns well.

Debugging Steps:

- Compare training and testing accuracy to detect overfitting.
- Visualize the decision tree to identify unnecessary complexity.
- Check data quality and remove duplicates or incorrect values.
- Analyze feature importance to identify irrelevant features.
- Perform cross-validation to evaluate model performance on different data splits.

Optimization Methods:

1. **Pruning:** Remove unnecessary branches to reduce model complexity.
2. **Cross-Validation:** Use techniques like k-fold cross-validation for reliable evaluation.
3. **Feature Selection:** Keep only the most important features.
4. **Limit Tree Depth:** Set a maximum depth to avoid overly complex trees.
5. **Increase Training Data:** More diverse data improves generalization.

Conclusion:
Applying pruning, feature selection, cross-validation, and controlling tree depth helps reduce overfitting and improves the model's performance on unseen data.

6. CNN Produces Unstable Results Across Training Epochs

Problem Analysis:
A Convolutional Neural Network (CNN) gives inconsistent accuracy and loss during training. This indicates unstable learning and poor convergence.

Possible Causes:

1. **Improper Data Augmentation:** Excessive or incorrect image transformations create inconsistent training samples.
2. **Vanishing Gradients:** Gradients become very small, slowing or stopping learning in deep networks.
3. **Batch Normalization Errors:** Incorrect placement or configuration of batch normalization layers causes unstable training.
4. **High Learning Rate:** Large updates make the model overshoot the optimal solution.
5. **Small Batch Size:** Produces noisy gradient estimates and unstable updates.

Debugging Strategies:

- Monitor training and validation loss curves.
- Verify that data augmentation is applied correctly.
- Check gradient values for vanishing or exploding gradients.
- Ensure batch normalization layers are correctly placed.
- Experiment with different learning rates and batch sizes.

Optimization Techniques:

1. **Proper Data Augmentation:** Apply realistic transformations such as rotation, flipping, and scaling.
2. **Batch Normalization:** Normalize activations to stabilize learning.
3. **Use ReLU Activation:** Reduces the vanishing gradient problem.
4. **Learning Rate Scheduling:** Gradually reduce the learning rate during training.
5. **Regularization:** Use dropout and L2 regularization to improve generalization.
6. **Early Stopping:** Stop training when validation performance stops improving.

Conclusion:
Stable CNN training can be achieved by using appropriate data augmentation, correct batch normalization, suitable learning rates, ReLU activation, and regularization techniques, resulting in improved accuracy and better generalization.